Name: Aarav Karani
Div: D12C ROLL NO: 18

TCS Self Learning
JFLAP Software

1) Deterministic Finite Automata

Deterministic Finite Automaton (DFA)

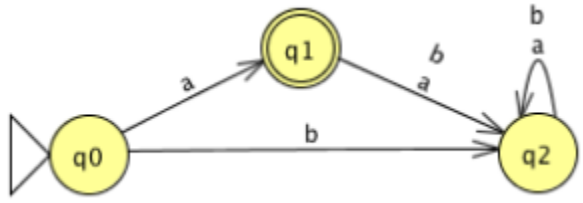
A Deterministic Finite Automaton (DFA) is a finite-state machine used to determine whether a string of symbols is accepted or rejected. For every input string, it follows exactly one possible sequence of transitions and always terminates with either an accepting or a rejecting result. The language recognized by a DFA M, denoted by $L(M)$, consists of all strings that are accepted by the machine.

Formal definition

A DFA is a 5-tuple (Q, Σ, δ, q_0, F), where:

- Q is a finite set of states
- Σ is a finite set of input symbols (the alphabet)
- δ: Q × Σ → Q is the transition function
- $q_0 \in Q$ is the start state
- $F \subseteq Q$ is the set of accept states

Example: This DFA over the alphabet {a, b} recognizes the language consisting of only the string “a”. It accepts “a” and rejects everything else.



State	a	b
q_0	q_1	q_2
q_1	q_2	q_2
q_2	q_2	q_2

Every state has exactly one transition for each symbol of the alphabet. Because the next state is uniquely determined, the automaton is deterministic.

Name: Aarav Karani
Div: D12C ROLL NO: 18

Example:

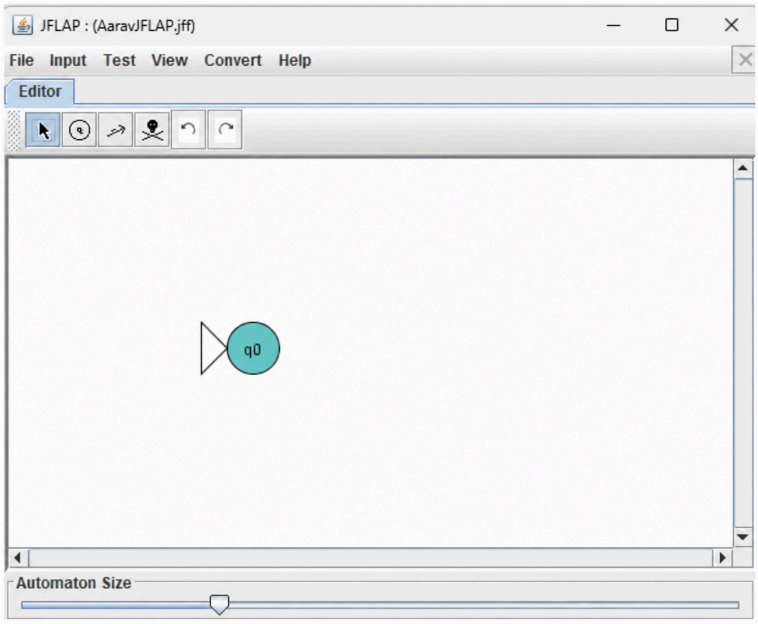
Consider a regular language L over the alphabet $\{a, b, c\}$ that contains all strings starting and ending with c. We can design a DFA M that accepts strings belonging to this language.

Examples of strings accepted by the language: $c, cc, ccc, cac, cabc, cabcbabc$.

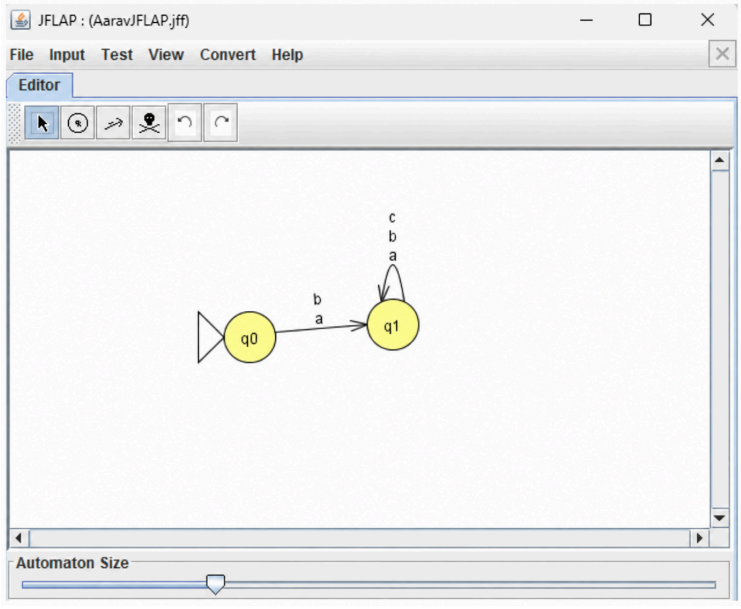
Examples of strings rejected by the language: $\lambda, a, ac, cb, ccca$.

Execution:

Step 1: Create initial state q0

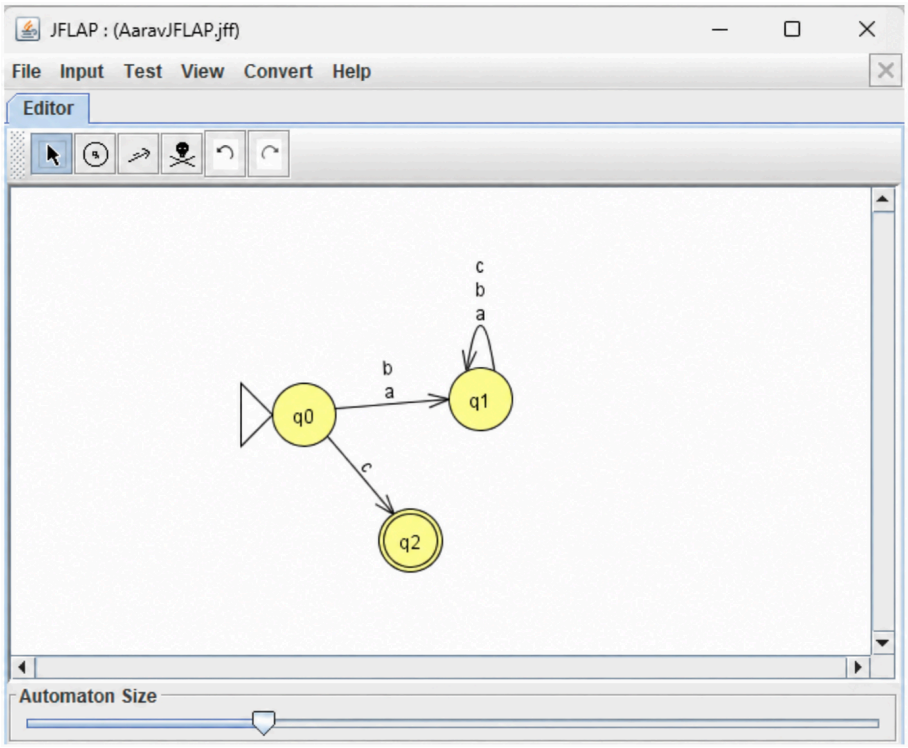


Step 2: Create a new state, q1, as the destination of a or b from q0 and which traps all subsequent substrings

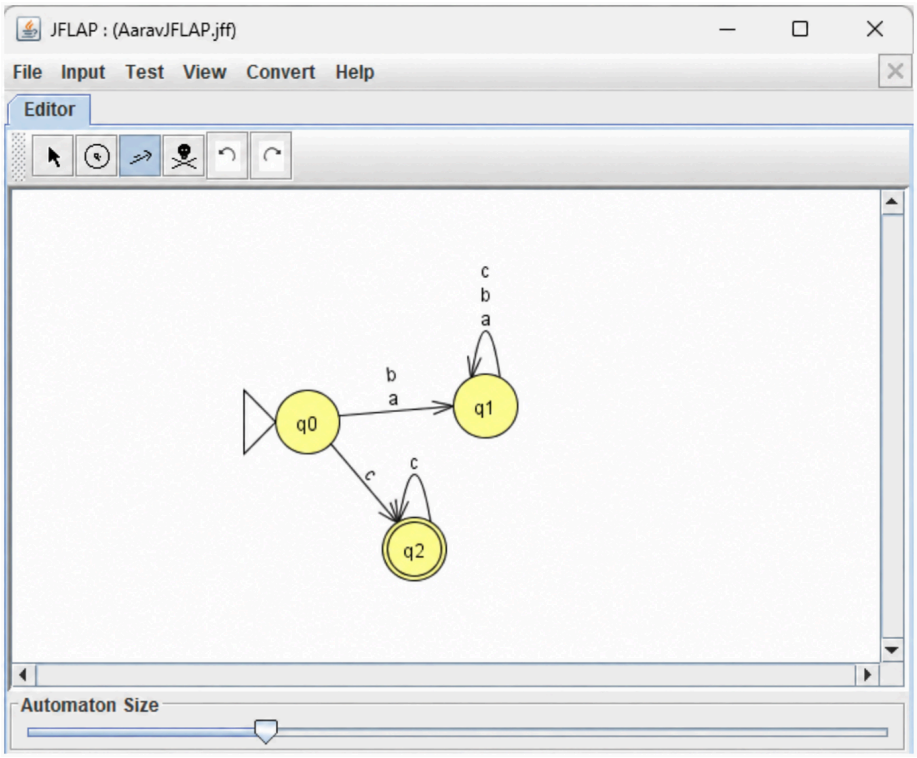


Name: Aarav Karani
Div: D12C ROLL NO: 18

Step 3: Create a new state, q2, as the destination of transition c from q0 and which is an accept state

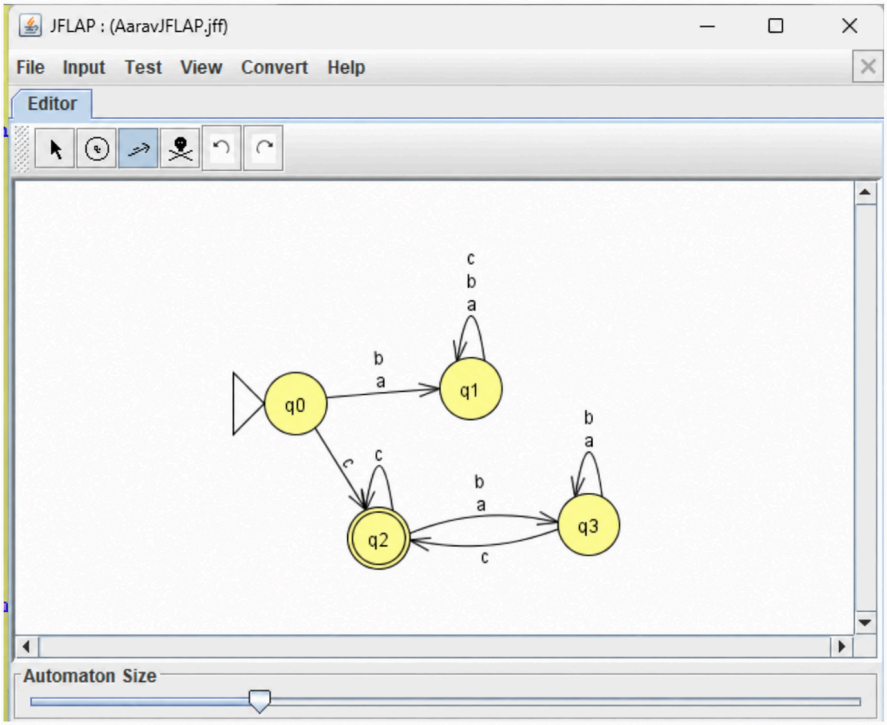


Step 4: Add a c transition to and from q2

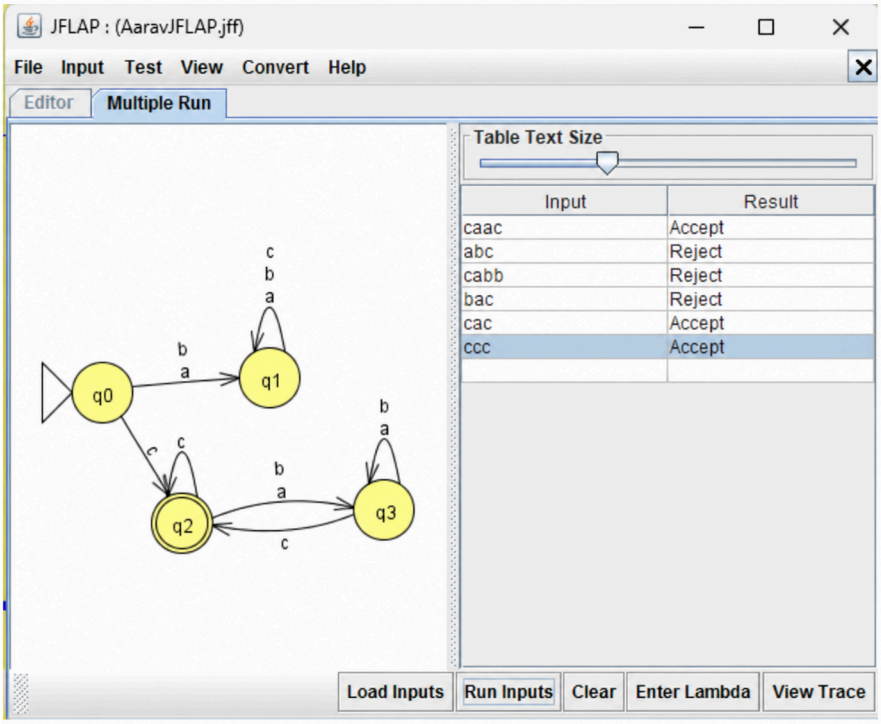


Name: Aarav Karani
Div: D12C ROLL NO: 18

Step 5: Add state q3 with a transition of a or b from q2. Add transitions to remain in q3 for a or b. Add transition from q3 back to q2 for c



Testing with inputs:



Therefore the DFA accepts all strings that start and end with c

Name: Aarav Karani
Div: D12C ROLL NO: 18

2) Non Deterministic Finite Automata

A Nondeterministic Finite Automaton (NFA) is a finite-state machine that accepts or rejects strings of symbols, similar to a DFA. However, an NFA can have multiple possible transitions or no transition for a given state and input symbol. An NFA N recognizes the language $L(N)$, which consists of all strings for which at least one possible computation path reaches an accepting state.

Example: An NFA over the alphabet $\{a, b\}$ can recognize all strings ending with “ab”. In state q_0, it continues reading the input and, whenever it encounters an a, it can move to q_1, guessing that this is the second-last symbol. It then expects a b to reach the accepting state q_2. Any additional symbol on that path prevents it from accepting the string.

Formal definition

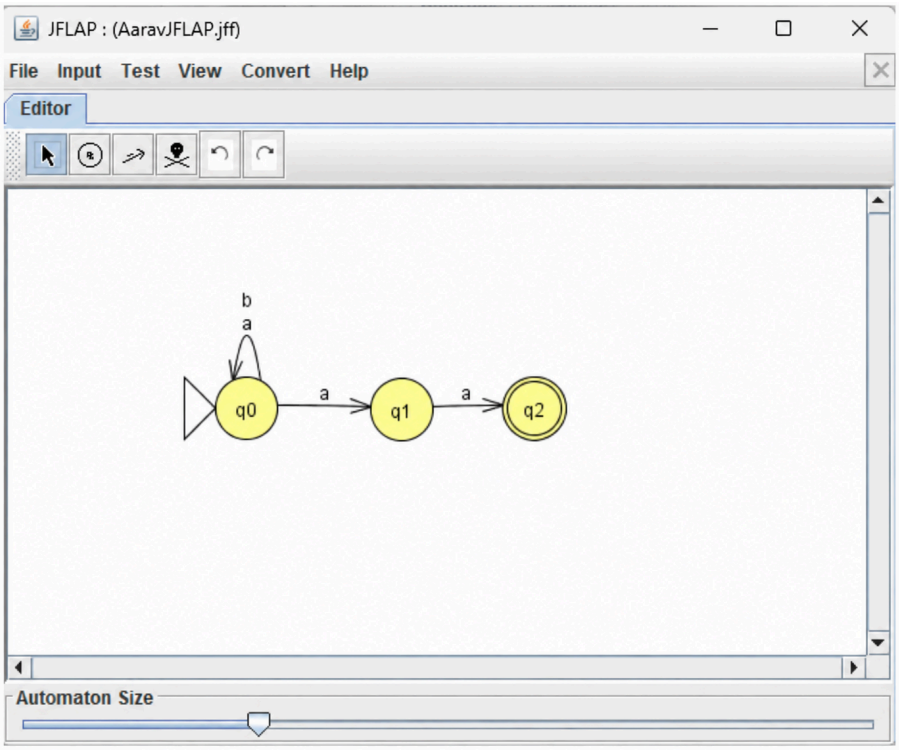
An NFA is a 5-tuple (Q, Σ, δ, q_0, F), where:

- Q is a finite set of states
- Σ is a finite set of input symbols (the alphabet)
- δ: Q × (Σ ∪ {ε}) → P(Q) is the transition function, which returns a set of states (possibly empty)
- q_0 ∈ Q is the start state
- F ⊆ Q is the set of accept states

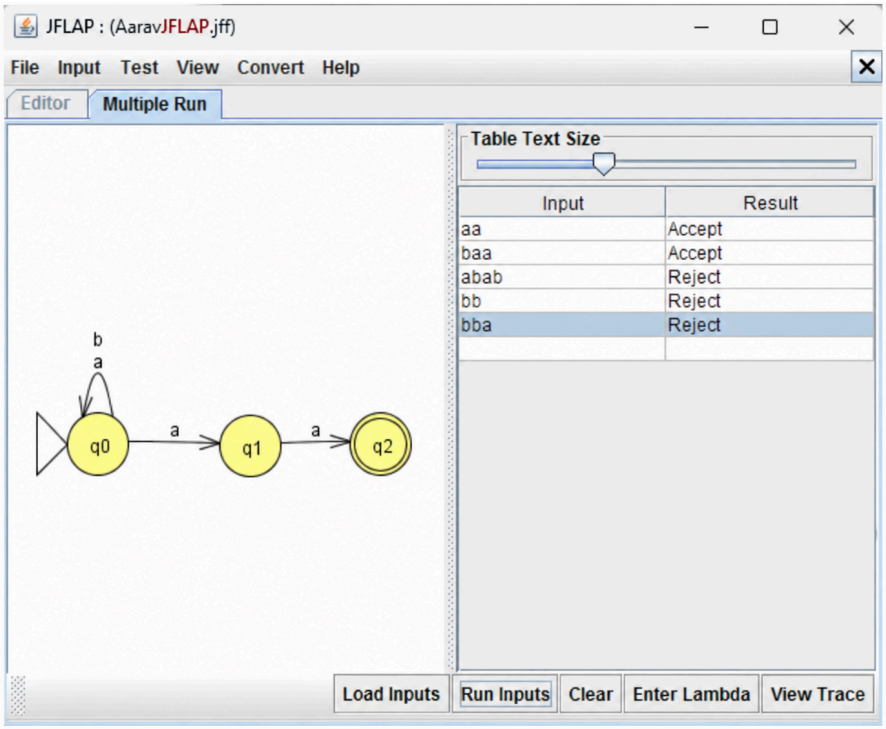
Example:
Define an NFA that recognizes the following language L over Σ = {a, b}:
L = { w | w ends with aa }.

Name: Aarav Karani
Div: D12C ROLL NO: 18

Execution:



Testing:



Therefore, this NFA successfully accepts all strings ending with aa.

Name: Aarav Karani
Div: D12C ROLL NO: 18

3) NFA to DFA Conversion

Every NFA can be converted into an equivalent DFA that accepts the same language. This conversion is performed using the subset construction method, in which each DFA state represents a set of NFA states. This allows the DFA to keep track of all possible states the NFA could reach after processing the input received so far.

Construction

Given an NFA $N = (Q, \Sigma, \delta, q_0, F)$, build a DFA $D = (Q', \Sigma, \delta', q_0', F')$, where:

- $Q' = P(Q)$: each DFA state is a subset of NFA states
- $q_0' = \{q_0\}$: the start state is the set containing only the NFA's start state
- $\delta'(S, a) = \cup\, \delta(q, a)$ for $q \in S$: on symbol a, move to the union of all states reachable from any state in S
- $F' = \{S \in Q' \mid S \cap F \neq \varnothing\}$: a DFA state accepts if it contains at least one NFA accept state

If the NFA has ε-transitions, take the ε-closure (all states reachable by ε-moves alone) of the start state and of every result of δ'.

Procedure

1. Start with the DFA state $\{q_0\}$ (or its ε-closure).
2. For each new DFA state S and each symbol a, compute $\delta'(S, a)$ as the union of the NFA transitions.
3. If the result is a new subset, add it as a state.
4. Repeat until no new subsets appear.
5. Mark every subset containing an NFA accept state as accepting.

Only subsets reachable from the start are needed, so the DFA is usually much smaller than the full 2^n.

Example

Convert the NFA from before (strings over {a, b} ending in "ab", start q_0, accept q_2):

State	a	b
q_0	$\{q_0, q_1\}$	$\{q_0\}$
q_1	$\varnothing$	$\{q_2\}$
q_2	$\varnothing$	$\varnothing$

Name: Aarav Karani
Div: D12C ROLL NO: 18

Starting from $\{q_0\}$:

- $\{q_0\}$ on a → $\{q_0, q_1\}$; on b → $\{q_0\}$
- $\{q_0, q_1\}$ on a → $\{q_0, q_1\} \cup \varnothing = \{q_0, q_1\}$; on b → $\{q_0\} \cup \{q_2\} = \{q_0, q_2\}$
- $\{q_0, q_2\}$ on a → $\{q_0, q_1\} \cup \varnothing = \{q_0, q_1\}$; on b → $\{q_0\} \cup \varnothing = \{q_0\}$

No new subsets appear, so the DFA has three states. Naming A = $\{q_0\}$, B = $\{q_0, q_1\}$, and C = $\{q_0, q_2\}$, the DFA is ({A, B, C}, {a, b}, δ′, A, {C}), with δ′ given by:

State	a	b
A	B	A
B	B	C
C	B	A

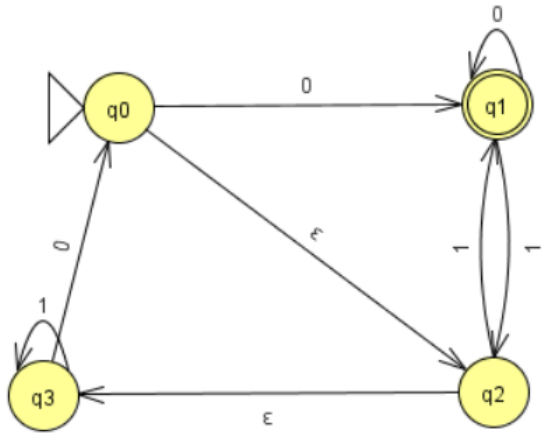
C is the only accepting state, since it is the only subset containing q_2. The other five subsets ($\varnothing$, $\{q_1\}$, $\{q_2\}$, $\{q_1, q_2\}$, $\{q_0, q_1, q_2\}$) are unreachable, so they are left out.

Key points

- The construction proves NFAs and DFAs recognize exactly the same languages, the regular languages.
- The DFA can need up to 2^n states for an n-state NFA, and some languages really do require this blowup.
- The empty set $\varnothing$ acts as the DFA's "dead" state when it is reachable.

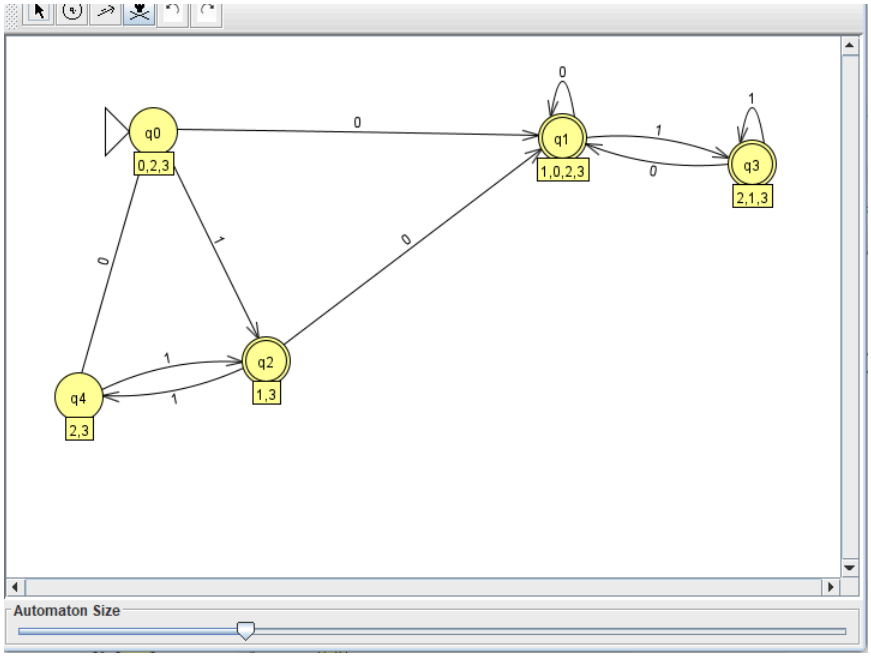
Example:

Convert the following NFA to an equivalent DFA



Name: Aarav Karani
Div: D12C ROLL NO: 18

Final DFA



4) DFA Minimization

Multiple DFAs can recognize the same language, but every regular language has a unique minimal DFA, which contains the smallest possible number of states. This DFA is unique except for the names assigned to its states. DFA minimization involves eliminating unreachable states and combining equivalent states.

Key Idea

Two states p and q are considered equivalent if every possible input string w, when processed from either state, produces the same acceptance or rejection result. Such states can be merged without affecting the language recognized by the DFA. However, if there exists a string w that produces different results from the two states, they are called distinguishable states.

Procedure (partition refinement)

1. Remove unreachable states. Delete any state that can't be reached from q_0.
2. Split by acceptance. Start with two groups: accepting states F and non-accepting states Q \ F.
3. Refine. Split a group if two of its states send some symbol a to different groups. Repeat until no group splits.
4. Merge. Each final group becomes one state of the minimal DFA. The start state is the group containing q_0, the accept states are the groups inside F, and transitions follow the group of the target.

Equivalently, the table-filling method marks pairs of states as distinguishable, starting with (accepting, non-accepting) pairs, then marking any pair that moves to an already marked pair on some symbol. Unmarked pairs at the end are equivalent.

Example

Take this DFA over {a, b} with start state 0 and F = {2, 3}:

State	a	b
0	1	2
1	1	3
2	1	3
3	1	2
4	0	4

- Step 1: State 4 is unreachable, so remove it.
- Step 2: The groups are {0, 1} (non-accepting) and {2, 3} (accepting).
- Step 3: In {0, 1}, on a both states go to 1 (same group), and on b they go to 2 and 3 (same group). In {2, 3}, on a both go to 1, and on b they go to 3 and 2 (same group). Nothing splits, so the partition is final.
- Step 4: Merge into P = {0, 1} and Q = {2, 3}.

The minimal DFA is ({P, Q}, {a, b}, δ, P, {Q}), with δ given by:

State	a	b
P	P	Q
Q	P	Q

It recognizes the strings that end in b. Five states reduced to two.

Already minimal

The 3-state DFA obtained by converting the NFA for strings ending in “ab” is already minimal. Initially, the states are divided into two groups: $\{A, B\}$ and $\{C\}$. When the input symbol b is processed, state A transitions to A, whereas state B transitions to C. Therefore, $\{A, B\}$ is split into separate groups, resulting in three individual state groups.

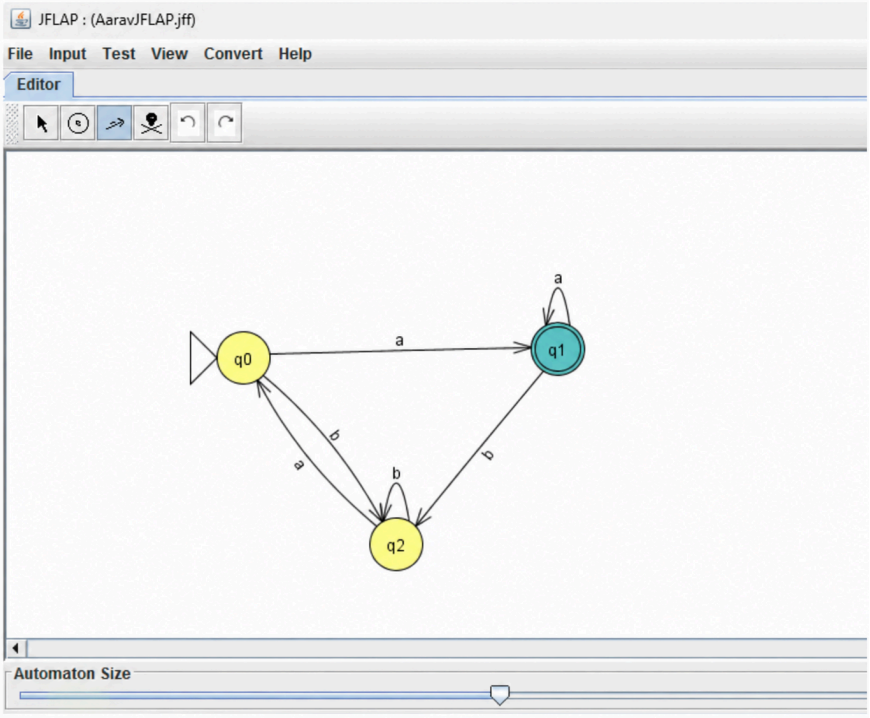
Name: Aarav Karani
Div: D12C ROLL NO: 18

Key Points

- DFA minimization reduces the number of states without changing the language recognized by the machine.
- Every regular language has a unique minimal DFA, up to state renaming. Two DFAs can be minimized and their structures compared to check whether they recognize the same language.
- The Myhill–Nerode theorem states that the number of states in a minimal DFA equals the number of equivalence classes of indistinguishable strings for that language.
- Hopcroft's algorithm minimizes a DFA efficiently in $O(n \log n)$ time, where n is the number of states.

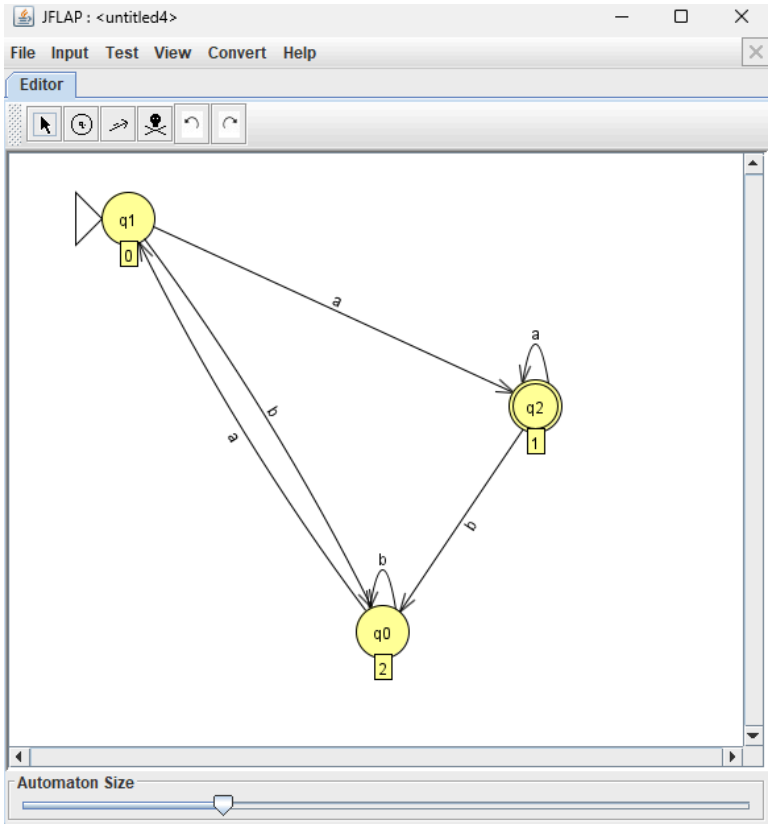
Example:
Consider the language L1 = {w ∈ Σ∗|w is λ or w ends in a}.

Execution:
DFA:



Minimized DFA:

Name: Aarav Karani
Div: D12C ROLL NO: 18



5) DFA to Regular Grammar Conversion

Every DFA can be transformed into an equivalent regular grammar that generates the same language accepted by the DFA. In this method, each state is represented by a non-terminal, and each transition is converted into a production rule. Thus, the grammar's derivation follows the same sequence of transitions as the DFA.

Regular Grammars

A right-linear grammar is a type of grammar in which productions have the following forms:

- $A \rightarrow aB$
- $A \rightarrow a$
- $A \rightarrow \varepsilon$

Here, A and B are non-terminals, a is a terminal, and ε represents the empty string. Each production generates at most one terminal followed by a non-terminal at the right end. Right-linear grammars generate exactly the class of regular languages.

Name: Aarav Karani
Div: D12C ROLL NO: 18

Construction

Given a DFA D = (Q, Σ, δ, q_0, F), build a grammar G = (V, Σ, P, S), where:

- **V = Q**: each state becomes a non-terminal
- **S = q_0**: the start state becomes the start symbol
- **Transition rule:** for each δ(p, a) = q, add the production **p → aq**
- **Accept rule:** for each p ∈ F, add the production **p → ε**

Reading a symbol corresponds to producing a terminal and moving to the next non-terminal. Reaching an accept state allows the derivation to end with ε.

Procedure

1. Name a non-terminal for each state, with the start state as the start symbol.
2. For every transition p —a→ q, write p → aq.
3. For every accept state p, write p → ε.
4. Optionally, drop any non-terminal that can never lead to a string (a dead or trap state) along with the productions that use it.

Example

Take the DFA from the NFA conversion (strings over {a, b} ending in “ab”), with start state A and accept state C:

State	a	b
A	B	A
B	B	C
C	B	A

The grammar has V = {A, B, C}, start symbol A, and these productions:

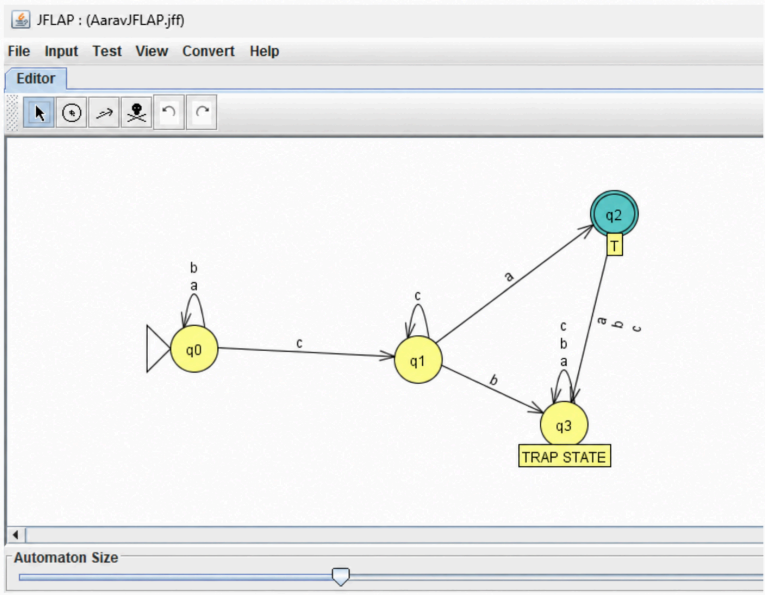
- A → aB | bA
- B → aB | bC
- C → aB | bA | ε

Derivation of “aab”: A ⇒ aB ⇒ aaB ⇒ aabC ⇒ aab

This matches the DFA’s run on “aab”: A →a→ B →a→ B →b→ C, which ends in the accept state C. A string like “aba” fails: the derivation reaches C after “ab”, but then the only way to continue is a → B, which cannot end in ε.

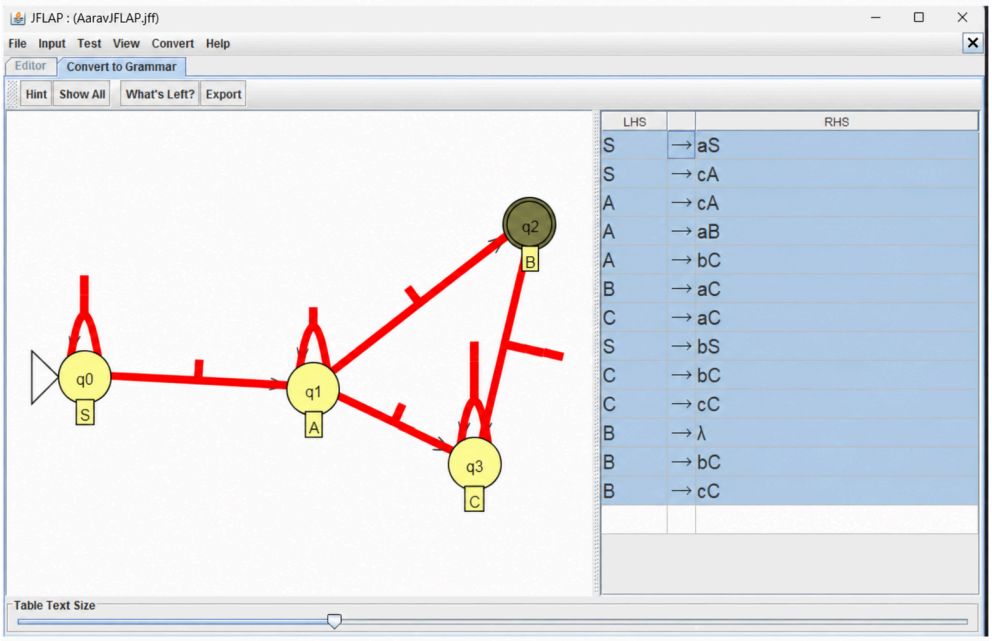
Name: Aarav Karani
Div: D12C ROLL NO: 18

Example;:
Consider this DFA



State q0 labeled R we have the following (→ is pronounced reduces to)
R → cS
R → aR
R → bR
S → aT
S → cS
T → null

Step 1:



Therefore DFA has been converted to Grammer

Name: Aarav Karani
Div: D12C ROLL NO: 18

6) DFA to Regular Expression Conversion

Every DFA can be converted into an equivalent regular expression that represents exactly the language accepted by the DFA. This is commonly done using the state elimination method, in which states are removed one by one and transition labels are replaced with regular expressions until only a single expression remains.

Key Idea

In this method, transitions are labeled with regular expressions instead of individual symbols. This representation is called a Generalized Nondeterministic Finite Automaton (GNFA). Whenever a state is eliminated, all paths passing through that state are replaced with direct transitions labeled by suitable regular expressions. This process preserves the language recognized by the automaton.

Procedure

1. Add a new start state S with an ε-edge to the old start state.
2. Add a new accept state F with ε-edges from every old accept state. These old states are no longer accepting.
3. Merge parallel edges. If several edges go from p to q, replace them with one edge labeled by the union ($R_1 \mid R_2 \mid \ldots$). If there is no edge, treat it as ∅.
4. Eliminate each original state q one at a time. For every pair of states p and r with an edge into q and an edge out of q, add or extend the edge p → r with the label R_pq (R_qq)* R_qr:
    - R_pq is the label from p to q
    - R_qq is the label of the loop on q (use ε if none)
    - R_qr is the label from q to r
    - If p → r already exists, union the new label with the old one.
5. Stop when only S and F remain. The label on the edge S → F is the regular expression.

Example

Take the DFA from the earlier sections (strings over {a, b} ending in “ab”), with start state A and accept state C:

State	a	b
A	B	A
B	B	C
C	B	A

After adding S and F, the edges are:

- S → A: ε
- A → A: b, A → B: a
- B → B: a, B → C: b
- C → B: a, C → A: b, C → F: ε

Eliminate A (loop b, so the middle term is b*):

- S → B: ε · b* · a = b*a
- C → B: a | (b · b* · a) = a | b⁺a = b*a

Eliminate C (no loop; the only edge in is from B on b, and the edges out are to B on b*a and to F on ε):

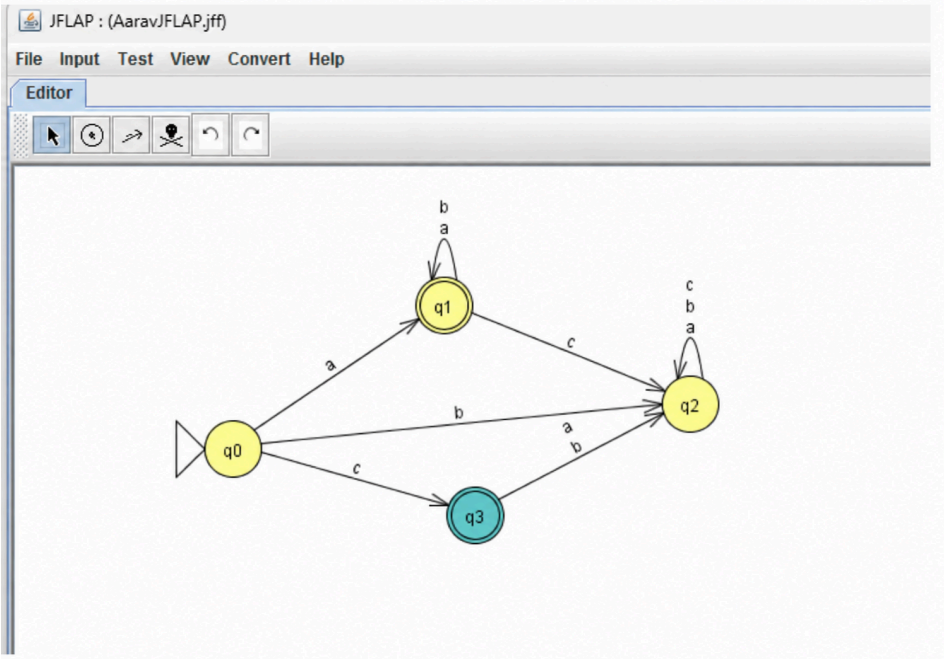
- B → B: a | (b · b*a) = b*a
- B → F: b

Eliminate B (loop b*a, so the middle term is (b*a)*):

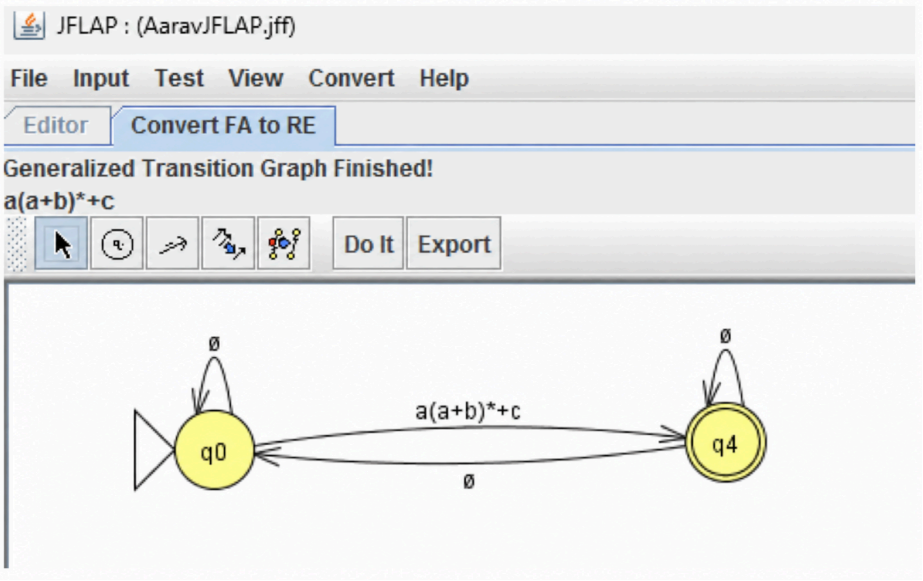
- S → F: b*a · (b*a)* · b = (b*a)⁺b

The regular expression is (b*a)⁺b. It is equivalent to the more familiar (a|b)*ab, since a string ends in “ab” exactly when it is a string ending in a (that is, (b*a)⁺) followed by b.

Example
Consider the following DFA



Name: Aarav Karani
Div: D12C ROLL NO: 18



Therefore we have successfully converted DFA to RE

7) Combining Automata

Regular languages are closed under union, intersection, difference, complement, concatenation, and star. Each has a construction on finite automata.

Product construction (union, intersection, difference)

Given DFAs $M_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$ and $M_2 = (Q_2, \Sigma, \delta_2, q_2, F_2)$ over the same alphabet, run both in parallel:

- States: $Q_1 \times Q_2$
- Start: (q_1, q_2)
- Transition: $\delta((p, q), a) = (\delta_1(p, a), \delta_2(q, a))$
- Accept states:
    - Union: $p \in F_1$ or $q \in F_2$
    - Intersection: $p \in F_1$ and $q \in F_2$
    - Difference $L_1 \setminus L_2$: $p \in F_1$ and $q \notin F_2$

Only accept set changes between operations.

Complement

Swap accepting and non-accepting states: $F' = Q \setminus F$. The DFA must be complete (a transition on every symbol from every state).

Name: Aarav Karani
Div: D12C ROLL NO: 18

ε-NFA constructions

- Union: add a new start state with ε-edges to both start states.
- Concatenation L_1L_2: add ε-edges from each accept state of M_1 to the start state of M_2. Only F_2 accepts.
- Star L*: add a new accepting start state with an ε-edge to the old start state, and ε-edges from each old accept state back to the old start state.

Example

Let M_1 accept an even number of a's (states E, O) and M_2 accept strings ending in b (states N, Y). The product starts at (E, N):

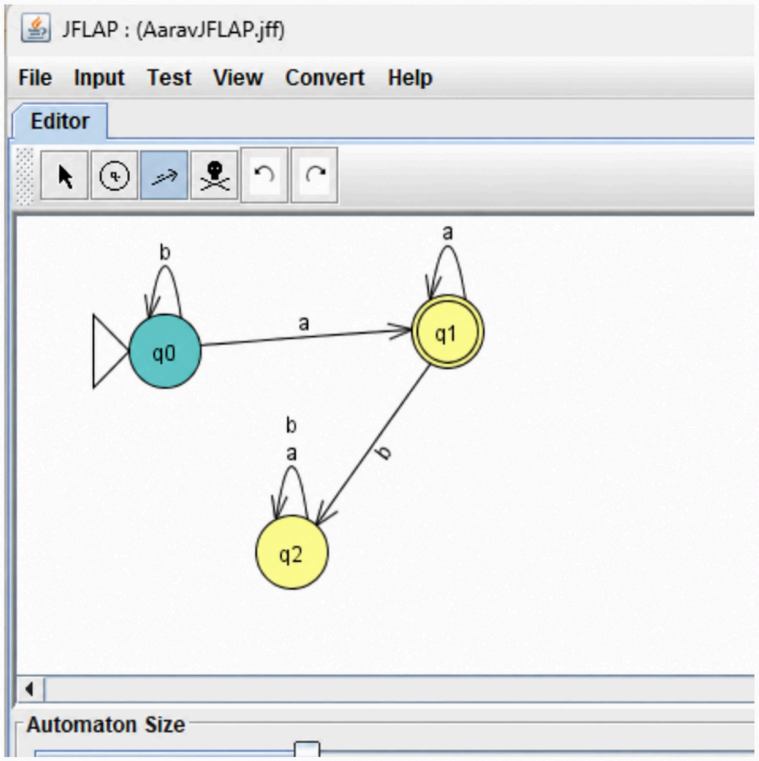
State	a	b
(E, N)	(O, N)	(E, Y)
(E, Y)	(O, N)	(E, Y)
(O, N)	(E, N)	(O, Y)
(O, Y)	(E, N)	(O, Y)

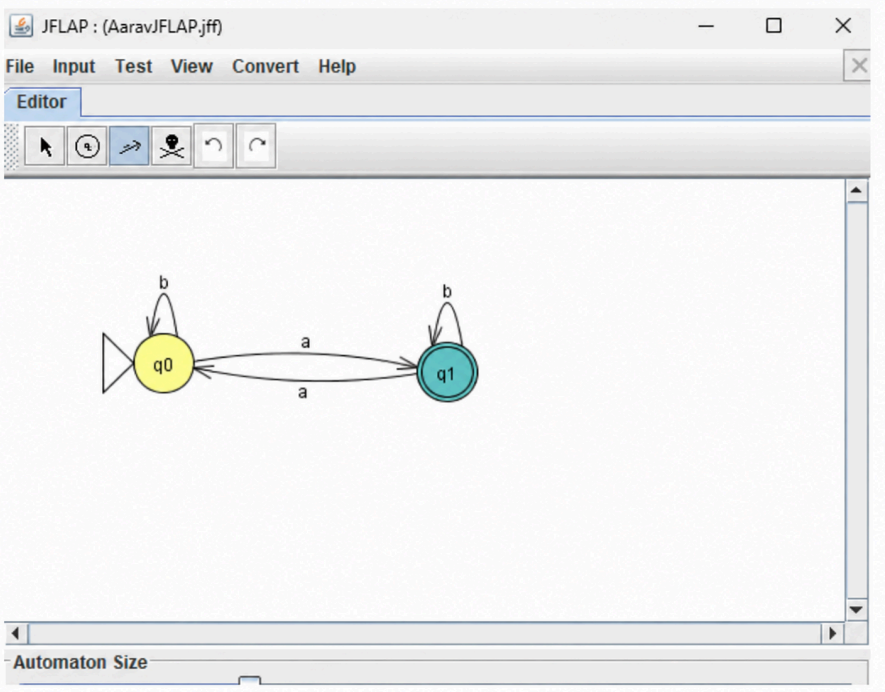
- Intersection: accept {(E, Y)}
- Union: accept {(E, N), (E, Y), (O, Y)}
- Difference $L_1 \setminus L_2$: accept {(E, N)}

Example:
In this Exercise, we will use the "Combine Automata" feature of JFLAP to find the intersection of two DFA. For the alphabet Σ = {a, b}, consider the languages
L1 = {w ∈ Σ∗|w has an odd number of a's}, and
L2 = {w ∈ Σ∗|w does not have ab as a substring}

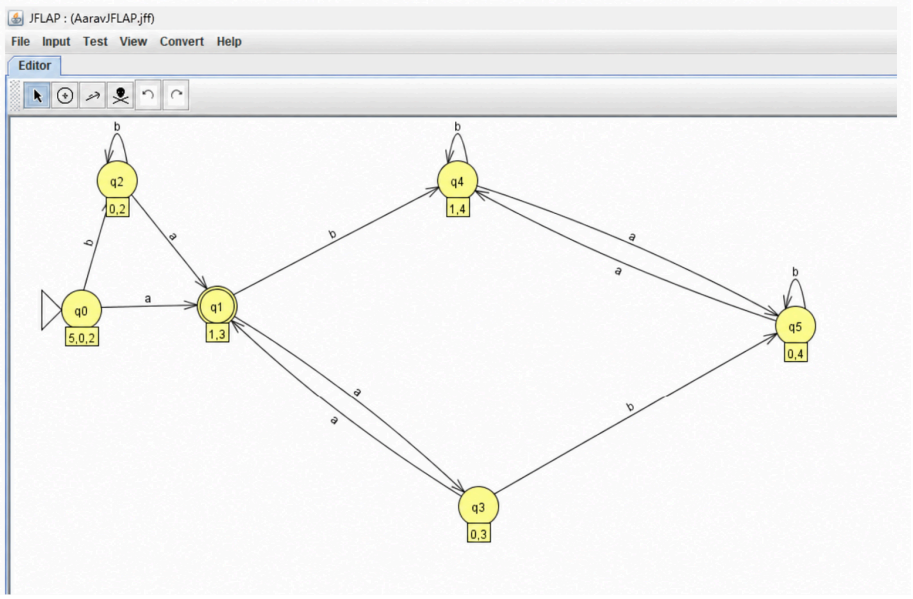
Name: Aarav Karani
Div: D12C ROLL NO: 18

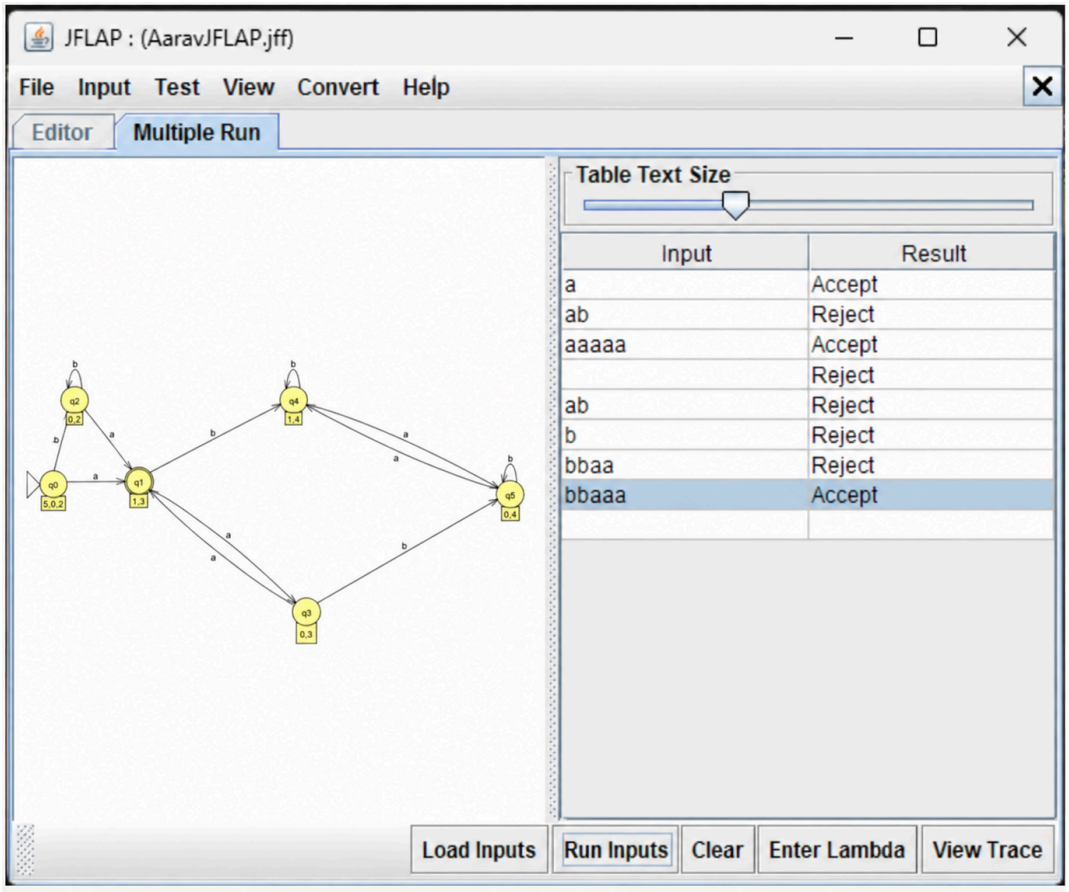
Step 1: Load the files odd-a.jflap and no-ab.jflap in two separate windows, as below





Name: Aarav Karani
Div: D12C ROLL NO: 18





Hence this DFA accepts all strings that have odd no. of a's and have no ab substring

Name: Aarav Karani
Div: D12C ROLL NO: 18

8) Regular Expression to DFA conversion
Regular Expression to DFA Conversion

Build an **ε-NFA** from the expression (**Thompson's construction**), then convert it to a DFA (**subset construction**). Minimizing afterwards is optional.

Step 1: Thompson's construction

Each piece has one start state and one accept state:

- **a (a symbol):** start —a→ accept
- **R | S:** a new start with ε-edges to the starts of R and S, and ε-edges from their accept states to a new accept state
- **RS:** an ε-edge from R's accept state to S's start
- **R*:** a new start and a new accept state. The new start has ε-edges to R's start and to the new accept. R's accept has ε-edges back to R's start and to the new accept.

Step 2: Subset construction

1. The start state is the **ε-closure** of the NFA's start state (all states reachable by ε-moves alone).
2. For each DFA state S and symbol a, compute **ε-closure(move(S, a))**.
3. Repeat until no new sets appear.
4. A set is accepted if it contains the NFA's accept state.

Example: (a|b)*ab

The Thompson NFA gives four DFA states:

- **A** = ε-closure of the start state
- **B** = the state reached after reading an a (the NFA is now ready to read b)
- **C** = the state reached after reading "ab" (contains the NFA accept state)
- **D** = the state reached after reading a b from A, B-less (same behavior as A)

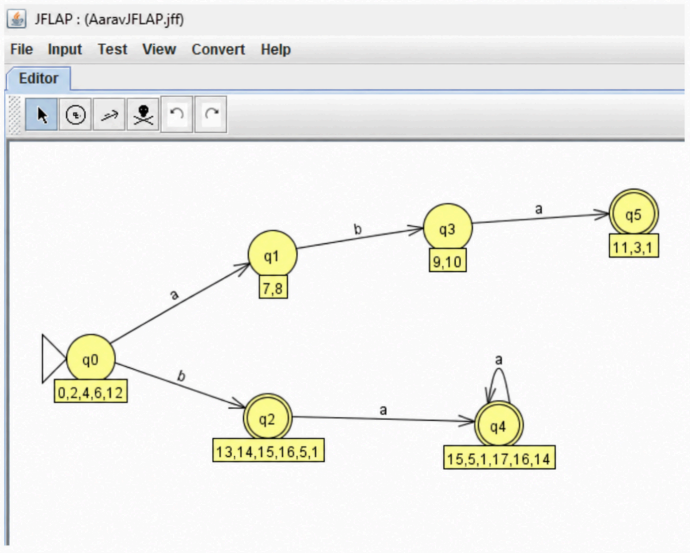
State	a	b
A	B	D
B	B	C
C	B	D
D	B	D

C is the only accepting state. A and D have identical rows, so minimization merges them:

State	a	b
A	B	A
B	B	C
C	B	A

This is the 3-state DFA for strings ending in “ab” from the earlier sections.

Example: Convert the following Regular Expression to a DFA aba ∪ ba∗



9) Mealy Machine

A Mealy machine is a finite-state machine that generates output based on its current state and the input symbol. Unlike a DFA, which accepts or rejects strings, a Mealy machine produces an output for every transition. Therefore, its output depends on both the current state and the input received.

Formal definition

A Mealy machine is a 6-tuple $(Q, \Sigma, \Delta, \delta, \lambda, q_0)$, where:

- **Q** is a finite set of states
- **Σ** is the input alphabet

- **Δ** is the output alphabet
- **δ: Q × Σ → Q** is the transition function
- **λ: Q × Σ → Δ** is the output function
- q_0 is the start state

There are no accepted states. The output has the same length as the input.

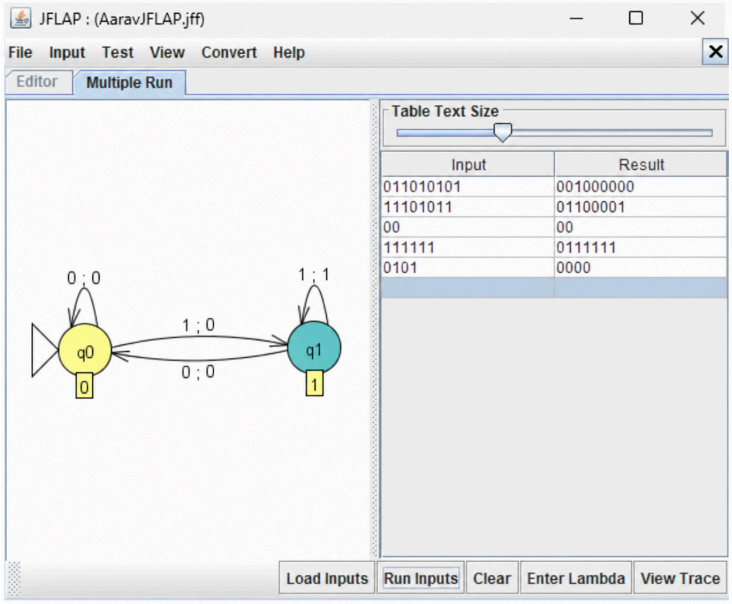
Example: detect “ab”

Over {a, b}, output 1 whenever the symbol just read completes “ab”, and 0 otherwise. The states are A (last symbol was not a) and B (last symbol was a). Each cell shows next state / output:

State	a	b
A (start)	B / 0	A / 0
B	B / 0	A / 1

On input “aab”: A —a/0→ B —a/0→ B —b/1→ A, so the output is **001**.

Example:
Construct a Mealy machine which takes a binary number and replaces the first 1 with a 0 from every substring starting with 1. For example, 0001001110 becomes 0000000110. This type of “bit stuffing” may be used in data transmission and telecommunications for run--length coding to limit the number of consecutive bits of the same value. A bit of the opposite value is inserted after the maximum allowed number of consecutive bit



10) Moore Machine

A Moore machine is a finite automaton that **produces output**, where the output depends **only on the current state**. Each state carries an output symbol, which is written whenever the machine enters that state.

Formal definition

A Moore machine is a 6-tuple $(Q, \Sigma, \Delta, \delta, \lambda, q_0)$, where:

- **Q** is a finite set of states
- **Σ** is the input alphabet
- **Δ** is the output alphabet
- **δ: Q × Σ → Q** is the transition function
- **λ: Q → Δ** is the output function, defined on states
- **q_0** is the start state

There are no accepted states. The output has length |input| + 1, since the start state also produces an output before any symbol is read.

Example: detect “ab”

Over {a, b}, output 1 when the last two symbols read are “ab”, and 0 otherwise. The states are A (last symbol was not a), B (last symbol was a), and C (just completed “ab”). Each state has an output, and each cell shows the next state:

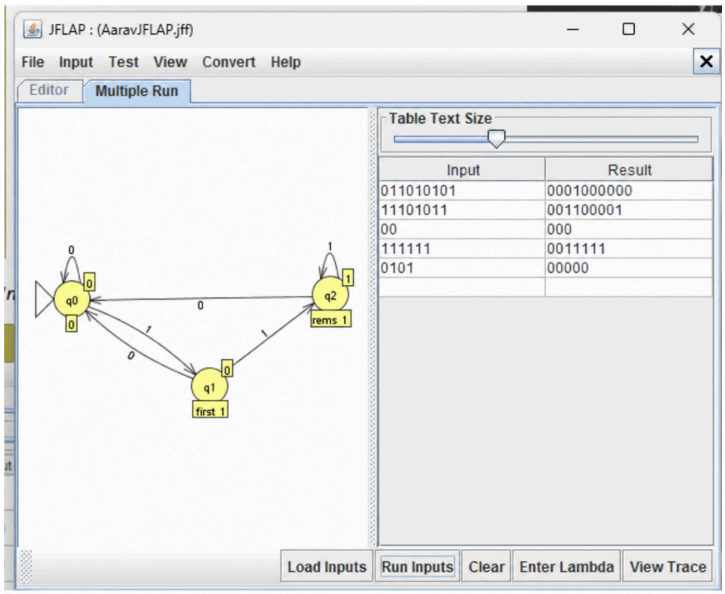
State	Output	a	b
A (start)	0	B	A
B	0	B	C
C	1	B	A

On input “aab”: A —a→ B —a→ B —b→ C, so the outputs are 0, 0, 0, 1, giving **0001**. The first 0 comes from the start state. Dropping it gives 001, the same as the Mealy machine.

Example:

Construct a Moore machine which takes a binary number and replaces the first 1 with a 0 from every substring starting with 1. For example, 0001001110 becomes 0000000110. This type of “bit stuffing” may be used in data transmission and telecommunications for

run--length coding to limit the number of consecutive bits of the same value. A bit of the opposite value is inserted after the maximum allowed number of consecutive bits.



Conclusion

The experiments were successfully carried out using JFLAP software to explore various concepts of automata theory. They covered the construction of DFA and NFA, NFA-to-DFA conversion, DFA minimization, and the conversion of DFAs into regular grammars and regular expressions. Operations on regular languages and the conversion of regular expressions into finite automata were also performed. Furthermore, Mealy and Moore machines were implemented to understand output generation based on transitions and states. Overall, these experiments provided practical understanding of finite-state machines, regular languages, and their implementation using JFLAP.